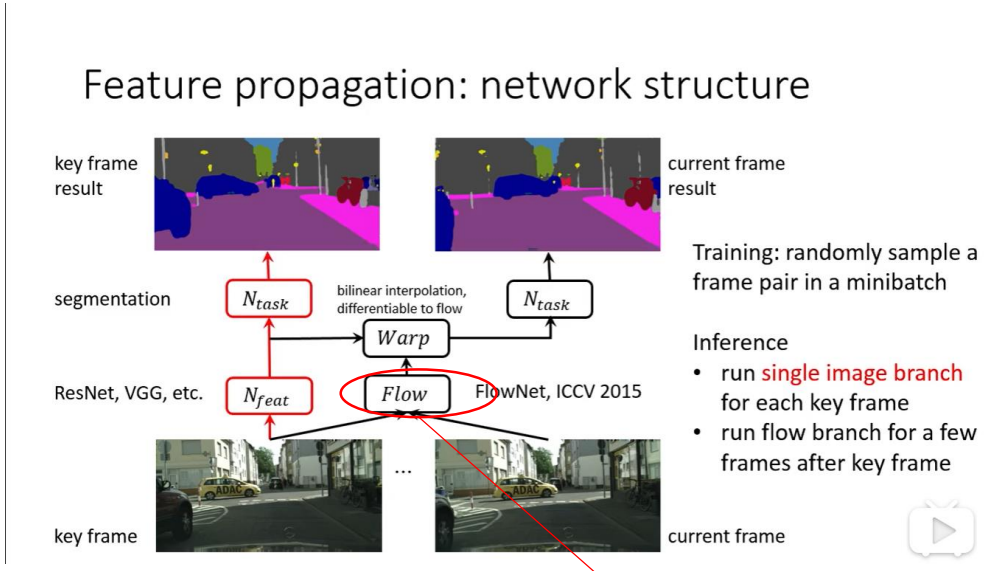
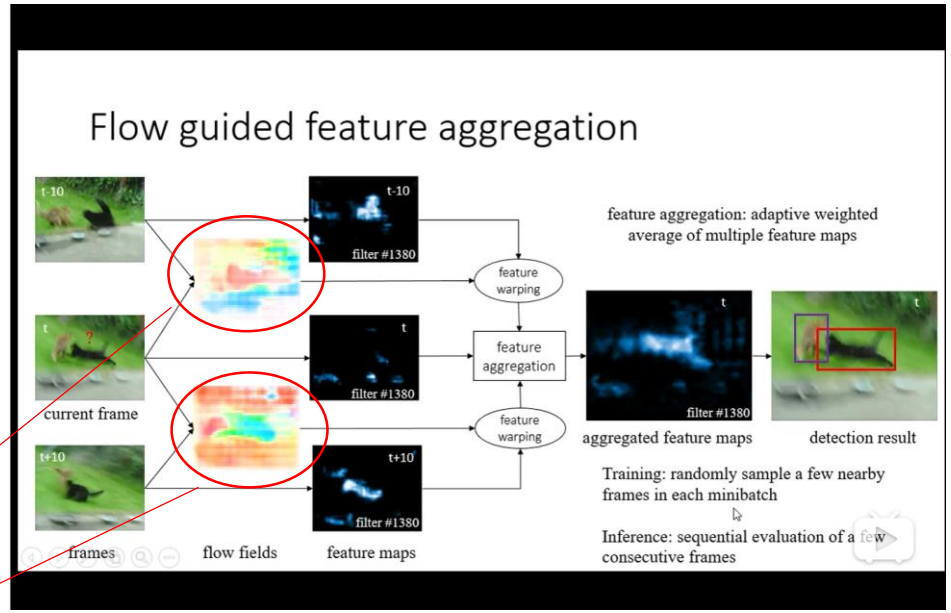
组会报告 Bilibili:av14789193

特征传播：
加速检测损失精度



特征聚合：
损失速度，加强检测精度



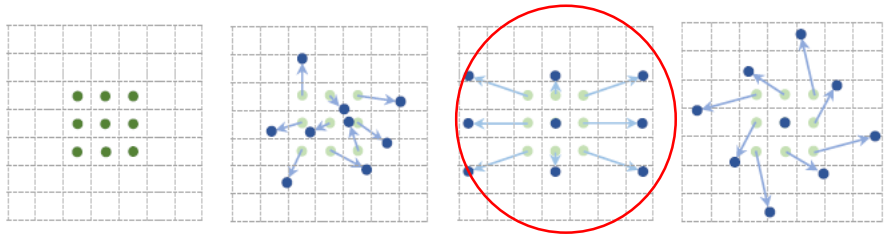
1.将flownet换成其他能有效捕捉位置关系的先验（烟雾运动）
2.头部网络可以共享

Deformable Convolutional Networks

- 传统的CNN只能靠一些简单的方法(比如max pooling)来适应物体的形变，如果形变的太厉害就无能为力了。为了改变这种情况专家们想了很多方法，最常见的有两种：
  1、使用大量的数据进行训练。比如用ImageNet数据集，再在其基础上做翻转等变化来扩展数据集，通俗地说就是通过穷举的方法使模型能够适应各种形状的物体，这种方法收敛较慢而且要设计复杂的网络结构才能达到理想的结果。
  2、设计一些特殊的算法来适应形变。比如SIFT，目标检测时用滑动窗口法来适应目标在不同位置上的分类也属于这类。

- 对第一种方法，如果用训练中没有遇到过的新形状物体(但同属于一类)来做测试，由于新形状没有训练过，会造成测试不准确，而且靠数据集来适应形变的训练过程太耗时，网络结构也必须设计的很复杂；对于第二种方法，如果物体的形状极其复杂，要设计出能适应这种复杂结构的算法就更困难了。

- 为了解决这个问题，提出了一种可形变的卷积结构(Deformable Convolution Networks)。它对感受野上的每一个点加一个偏移量，偏移的大小是通过学习得来的，偏移后感受野不再是个正方形，而是和物体的实际形状相匹配。这么做的好处就是无论物体怎么形变，卷积的区域始终覆盖在物体形状的周围。



(a) (b) (c) (d)

Figure 1: Illustration of the sampling locations in 3×3 standard and deformable convolutions. (a) regular sampling grid (green points) of standard convolution. (b) deformed sampling locations (dark blue points) with augmented offsets (light blue arrows) in deformable convolution. (c)(d) are special cases of (b), showing that the deformable convolution generalizes various transformations for scale, (anisotropic) aspect ratio and rotation.

Deformable Convolution

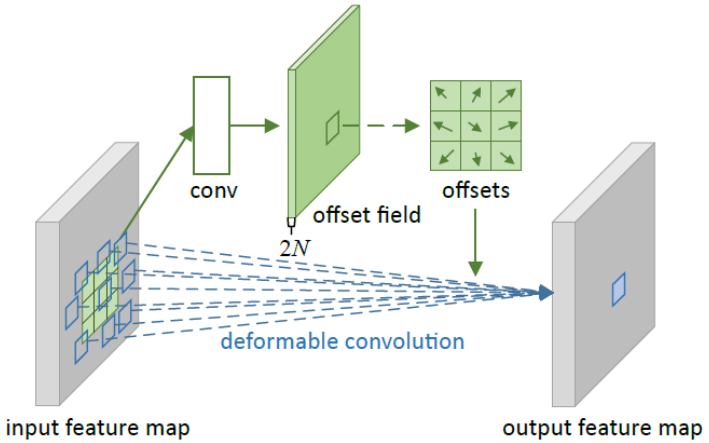


Figure 2: Illustration of 3 × 3 deformable convolution.

传统的卷积结构可以定义成公式1，其中p_n是卷积输出每一个点相对感受野上的每一个点的偏移量，取整数。

$$\mathbf{y}(\mathbf{p}_0) = \sum_{\mathbf{p}_n \in \mathcal{R}} \mathbf{w}(\mathbf{p}_n) \cdot \mathbf{x}(\mathbf{p}_0 + \mathbf{p}_n), \qquad (1)$$

公式1

采用全新的可变形卷积后要在公式1基础上给每个点再增加一个偏移量Δp_n，这个新的偏移量是由另一个卷积得出，所以一般是小数，见公式2。

$$\mathbf{y}(\mathbf{p}_0) = \sum_{\mathbf{p}_n \in \mathcal{R}} \mathbf{w}(\mathbf{p}_n) \cdot \mathbf{x}(\mathbf{p}_0 + \mathbf{p}_n + \Delta\mathbf{p}_n). \qquad (2)$$

公式2

公式2中的$x(p_0+p_n+\Delta p_n)$的取值位置非整数，并不对应feature map上实际存在的点，所以必须用插值来得到，如果采用双线性插值的方法，$x(p_0+p_n+\Delta p_n)$可以变成公式3。其中$x(q)$表示feature map上所有整数位置上的点的取值，$x(p)=x(p_0+p_n+\Delta p_n)$表示加上偏移后所有小数位置的点的取值。

$$\begin{aligned} x(\mathrm{p}) &= \sum_q G(\mathrm{q},\mathrm{p}) \cdot x(\mathrm{q}) \\ &= \sum_q g(\mathrm{q_x},\mathrm{p_x}) \cdot g(\mathrm{q_y},\mathrm{p_y}) \cdot x(\mathrm{q}) \\ &= \sum_q \max(0, 1-|\mathrm{q_x}-\mathrm{p_x}|) \cdot \max(0, 1-|\mathrm{q_y}-\mathrm{p_y}|) \cdot x(\mathrm{q}) \end{aligned}$$

$$
\begin{aligned}
x(\mathrm{p}) &= \sum_{q} G(\mathrm{q},\mathrm{p}) \cdot x(\mathrm{q}) \\
&= \sum_{q} g(\mathrm{q_x},\mathrm{p_x}) \cdot g(\mathrm{q_y},\mathrm{p_y}) \cdot x(\mathrm{q}) \\
&= \sum_{q} \max(0,1-|\mathrm{q_x}-\mathrm{p_x}|) \cdot \max(0,1-|\mathrm{q_y}-\mathrm{p_y}|) \cdot x(\mathrm{q})
\end{aligned}
$$

公式3

公式3是怎么推导出的呢？先拿最简单的例子来做说明，假设feature map只有4个点，如图2，则其中插入一个点$P(x,y)$的值可以用公式4来得到，这就是双线形插值的标准公式，对于相邻的点来说x_1-x_0=1、y_1-y_0=1，所以可以继续简化成公式5。如果feature map上有q个点，公式5就等价于前面的公式3。

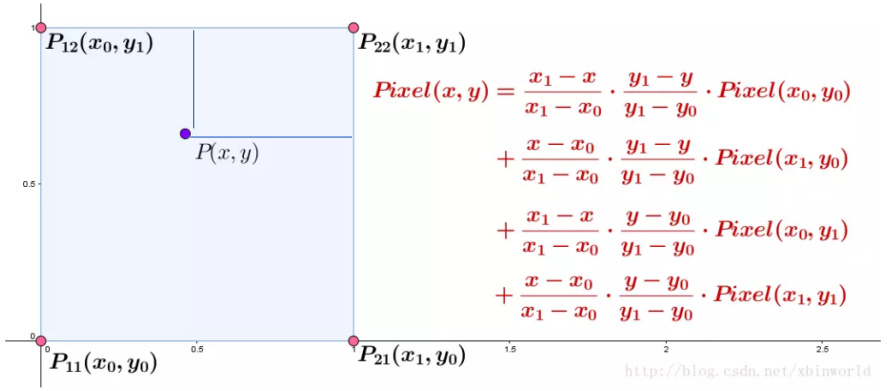


图2：双线形插值的过程

$$
\begin{aligned}
Pixel(x,y) = {} & \frac{x_1-x}{x_1-x_0} \cdot \frac{y_1-y}{y_1-y_0} \cdot Pixel(x_0,y_0) \\
& + \frac{x-x_0}{x_1-x_0} \cdot \frac{y_1-y}{y_1-y_0} \cdot Pixel(x_1,y_0) \\
& + \frac{x_1-x}{x_1-x_0} \cdot \frac{y-y_0}{y_1-y_0} \cdot Pixel(x_0,y_1) \\
& + \frac{x-x_0}{x_1-x_0} \cdot \frac{y-y_0}{y_1-y_0} \cdot Pixel(x_1,y_1)
\end{aligned}
$$

公式4

$$
\begin{aligned}
Pixel(\mathrm{x},\mathrm{y}) = {} & (\mathrm{x_1}-\mathrm{x}) \cdot (\mathrm{y_1}-\mathrm{y}) \cdot Pixel(\mathrm{x_0},\mathrm{y_0}) \\
& + (\mathrm{x}-\mathrm{x_0}) \cdot (\mathrm{y_1}-\mathrm{y}) \cdot Pixel(\mathrm{x_1},\mathrm{y_0}) \\
& + (\mathrm{x_1}-\mathrm{x}) \cdot (\mathrm{y}-\mathrm{y_0}) \cdot Pixel(\mathrm{x_0},\mathrm{y_1}) \\
& + (\mathrm{x}-\mathrm{x_0}) \cdot (\mathrm{y}-\mathrm{y_0}) \cdot Pixel(\mathrm{x_1},\mathrm{y_1})
\end{aligned}
$$

公式5

Deformable RoI Pooling

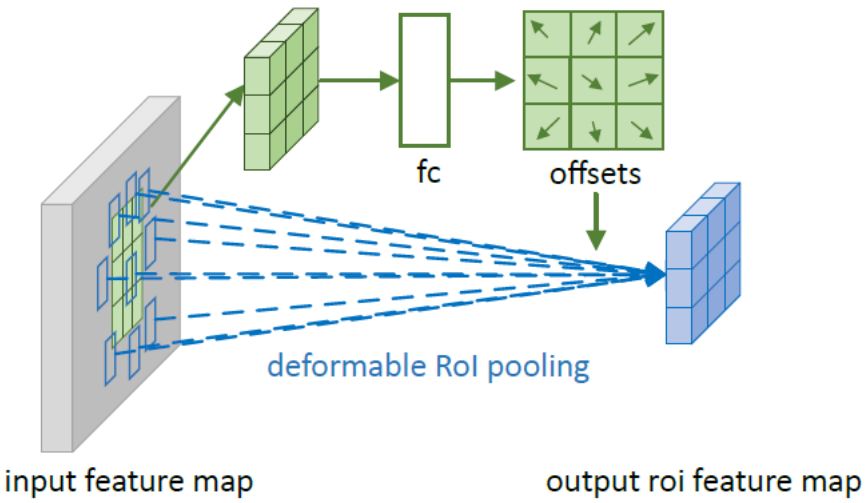


Figure 3: Illustration of 3×3 deformable RoI pooling.

传统的ROI Pooling可以用公式6来表示。整个ROI被分为k*k个bin，每个bin左上角的坐标是p_0，p是bin中每个点相对于p_0的坐标偏移量，n_{ij}是第ij个bin中的点数。

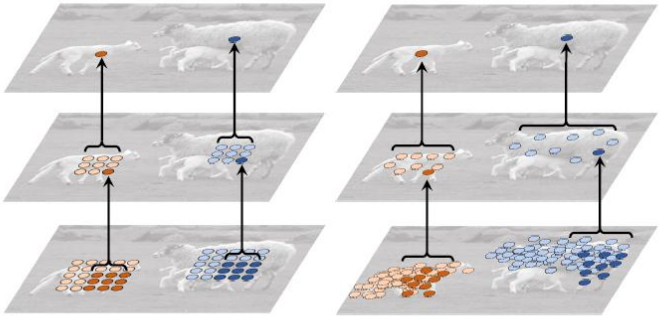
$$\mathbf{y}(i,j) = \sum_{\mathbf{p} \in bin(i,j)} \mathbf{x}(\mathbf{p}_0 + \mathbf{p})/n_{ij},$$

公式6

$$\mathbf{y}(i,j) = \sum_{\mathbf{p} \in bin(i,j)} \mathbf{x}(\mathbf{p}_0 + \mathbf{p} + \Delta\mathbf{p}_{ij})/n_{ij}.$$

公式7

将可变形的卷积套用到传统的ROI Pooling上可以得到可变形的ROI Pooling的公式7。其中Δp_{ij}是每个bin的偏移量，要注意这个偏移量是针对整个bin的，也就是说一个bin中的每一个点该值都相同。



(a) standard convolution (b) deformable convolution

Figure 5: Illustration of the fixed receptive field in standard convolution (a) and the adaptive receptive field in deformable convolution (b), using two layers. Top: two activation units on the top feature map, on two objects of different scales and shapes. The activation is from a 3×3 filter. Middle: the sampling locations of the 3×3 filter on the preceding feature map. Another two activation units are highlighted. Bottom: the sampling locations of two levels of 3×3 filters on the preceding feature map. Two sets of locations are highlighted, corresponding to the highlighted units above.

- 可以看到经过两层的传统卷积和两层的Deformable卷积的对比结果。左侧的传统卷积单个目标共覆盖了5*5=25个采样点，感受野始终是固定不变的方形；右侧的可变形卷积因为感受野的每一个点都有偏移量，造成卷积核在图片上滑动时对应的感受野的点不会重复选择，这意味着会采样9*9=81个采样点，比传统卷积更多。

Figure 6: Each image triplet shows the sampling locations ($9^3 = 729$ red points in each image) in three levels of 3×3 deformable filters (see Figure 5 as a reference) for three activation units (green points) on the background (left), a small object (middle), and a large object (right), respectively.

- 采样点都集中在了目标点周围

结果

usage of deformable convolution (# layers)	DeepLab		class-aware RPN		Faster R-CNN		R-FCN	
	mIoU@V (%)	mIoU@C (%)	mAP@0.5 (%)	mAP@0.7 (%)	mAP@0.5 (%)	mAP@0.7 (%)	mAP@0.5 (%)	mAP@0.7 (%)
none (0, baseline)	69.7	70.4	68.0	44.9	78.1	62.1	80.0	61.8
res5c (1)	73.9	73.5	73.5	54.4	78.6	63.8	80.6	63.0
res5b,c (2)	74.8	74.4	74.3	56.3	78.5	63.3	81.0	63.8
res5a,b,c (3, default)	**75.2**	**75.2**	74.5	57.2	78.6	63.3	81.4	64.7
res5 & res4b22,b21,b20 (6)	74.8	75.1	**74.6**	**57.7**	**78.7**	**64.0**	**81.5**	**65.4**

deformation modules	DeepLab mIoU@V / @C	class-aware RPN mAP@0.5 / @0.7	Faster R-CNN mAP@0.5 / @0.7	R-FCN mAP@0.5 / @0.7
atrous convolution (2,2,2) (default)	69.7 / 70.4	68.0 / 44.9	78.1 / 62.1	80.0 / 61.8
atrous convolution (4,4,4)	73.1 / 71.9	72.8 / 53.1	78.6 / 63.1	80.5 / 63.0
atrous convolution (6,6,6)	73.6 / 72.7	73.6 / 55.2	78.5 / 62.3	80.2 / 63.5
atrous convolution (8,8,8)	73.2 / 72.4	73.2 / 55.1	77.8 / 61.8	80.3 / 63.2
deformable convolution	**75.3 / 75.2**	**74.5 / 57.2**	78.6 / 63.3	81.4 / 64.7
deformable RoI pooling	N.A	N.A	78.3 / 66.6	81.2 / 65.0
deformable convolution & RoI pooling	N.A	N.A	**79.3 / 66.9**	**82.6 / 68.5**

Table 3: Evaluation of our deformable modules and atrous convolution, using ResNet-101.

layer	small	medium	large	background
	mean ± std			
res5c	5.3 ± 3.3	5.8 ± 3.5	8.4 ± 4.5	6.2 ± 3.0
res5b	2.5 ± 1.3	3.1 ± 1.5	5.1 ± 2.5	3.2 ± 1.2
res5a	2.2 ± 1.2	2.9 ± 1.3	4.2 ± 1.6	3.1 ± 1.1

Table 2: Statistics of effective dilation values of deformable convolutional filters on three layers and four categories. Similar as in COCO [39], we divide the objects into three categories equally according to the bounding box area. Small: area $< 96^2$ pixels; medium: $96^2 <$ area $< 224^2$; large: area $> 224^2$ pixels.

1) the receptive field sizes of deformable filters are correlated with object sizes, indicating that the deformation is effectively learned from image content;
2) the filter sizes on the background region are between those on medium and large objects, indicating that a relatively large receptive field is necessary for recognizing the background regions.

参数量增加并不明显

method	# params	net. forward (sec)	runtime (sec)
DeepLab@C	46.0 M	0.610	0.650
Ours	46.1 M	0.656	0.696
DeepLab@V	46.0 M	0.084	0.094
Ours	46.1 M	0.088	0.098
class-aware RPN	46.0 M	0.142	0.323
Ours	46.1 M	0.152	0.334
Faster R-CNN	58.3 M	0.147	0.190
Ours	59.9 M	0.192	0.234
R-FCN	47.1 M	0.143	0.170
Ours	49.5 M	0.169	0.193